TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

01

// Unidad I · Fundamentos de Java

PRIMEROS PASOS CON JAVA

DEL JDK A TU PRIMER PROGRAMA

EN ESTA LECTURA

01 ¿Qué es Java?
02 Instalar el JDK en Windows
03 Configurar JAVA_HOME y Path
04 Visual Studio Code para Java
05 Variables y tipos de datos
06 Estructuras de control
07 Para practicar



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco “El Chele” Moreno

Fundamentos de Desarrollo
Backend con Java
Octubre 2026

CAPÍTULO 01

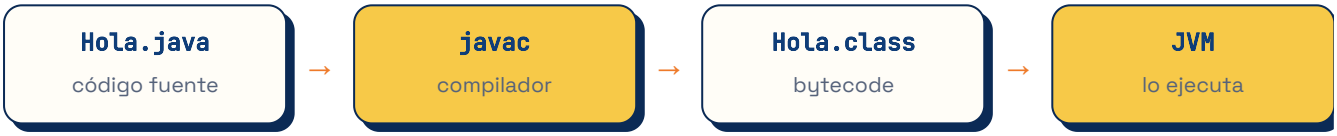
1 ¿QUÉ ES JAVA?

> Java es un lenguaje de programación de propósito general, orientado a objetos, creado para que un mismo programa funcione en cualquier sistema operativo sin reescribirlo.

Java nació en **1995** en Sun Microsystems, de la mano de un equipo liderado por **James Gosling**. Hoy lo mantiene **Oracle** junto con una enorme comunidad abierta (OpenJDK). Tres décadas después sigue entre los lenguajes más usados del mundo, sobre todo en el **desarrollo backend**: la parte de una aplicación que no se ve, la que procesa datos, aplica reglas de negocio y se comunica con las bases de datos.

“Escribe una vez, ejecuta en cualquier lugar”

La gran idea de Java es que tu código no se traduce directamente al lenguaje de una computadora específica. Se compila a un formato intermedio llamado **bytecode** (archivos `.class`), y ese bytecode lo ejecuta la **Máquina Virtual de Java (JVM)**, que existe para Windows, Linux, macOS y más.



JDK, JRE y JVM: ¿cuál es cuál?

Son tres siglas que vas a escuchar todo el curso. La forma más fácil de entenderlas es como cajas, una dentro de otra:

JDK • JAVA DEVELOPMENT KIT
Lo que instala el **programador**. Incluye el compilador `javac`, herramientas de desarrollo y todo lo de abajo.

JRE • JAVA RUNTIME ENVIRONMENT
El entorno para **ejecutar** programas: bibliotecas estándar de Java + la JVM.

JVM • JAVA VIRTUAL MACHINE
El “motor” que lee el bytecode y lo convierte en instrucciones que entiende tu computadora.

DATO

Para programar necesitas el **JDK**. Si solo instalas el JRE podrás ejecutar programas, pero no compilarlos.

¿Dónde se usa Java?

- **Backend y APIs:** sistemas bancarios, tiendas en línea y plataformas con frameworks como **Spring Boot**, que veremos en la Unidad II.

- **Aplicaciones Android:** durante años fue el lenguaje oficial de Android y sigue siendo muy usado.
- **Sistemas empresariales** de gran escala, big data (Hadoop, Kafka) y hasta videojuegos como *Minecraft*.

Características clave

Característica	¿Qué significa para ti?
Orientado a objetos	Organizas el código en clases y objetos que representan cosas del mundo real.
Fuertemente tipado	Cada variable tiene un tipo fijo; el compilador detecta muchos errores antes de ejecutar.
Multiplataforma	El mismo `.class` corre en cualquier sistema que tenga JVM.
Memoria automática	El *Garbage Collector* libera la memoria que ya no usas.
Versiones LTS	Cada dos años sale una versión con soporte extendido. En este curso usamos **Java 25 LTS**.

CAPÍTULO 02

2 INSTALAR EL JDK EN WINDOWS

> Usaremos **Eclipse Temurin**, una distribución gratuita y de código abierto del JDK, mantenida por la fundación Eclipse (proyecto Adoptium).

1. **Ve al sitio de descarga.** Abre el navegador y entra a `adoptium.net`. Haz clic en **Latest LTS Release** o ve a la sección **Temurin → Releases**.
2. **Elige tu sistema.** Selecciona: Sistema operativo **Windows**, Arquitectura **x64**, Tipo **JDK** y Versión **25 - LTS**.
3. **Descarga el instalador `.msi`.** Es el archivo más cómodo porque trae un asistente de instalación. (El `.zip` es para instalaciones manuales).
4. **Ejecuta el instalador.** Haz doble clic en el archivo descargado, acepta la licencia y pulsa **Next**.
5. **Revisa las opciones.** En la pantalla **Custom Setup** verás opciones como **Add to PATH** y **Set JAVA_HOME variable**. Puedes activarlas, pero en el siguiente capítulo aprenderás a hacerlo a mano, que es lo que un desarrollador debe saber.
6. **Termina la instalación.** Pulsa **Install**, acepta el permiso de administrador y luego **Finish**.

¿Dónde quedó instalado?

Por defecto, Temurin se instala en una carpeta parecida a esta (el número exacto depende de la versión que descargaste):

```
C:\Program Files\Eclipse Adoptium\jdk-25.0.x-hotspot\
```

Abre esa carpeta en el Explorador de archivos. Dentro verás la subcarpeta `bin`, donde viven los programas `java.exe` y `javac.exe`. **Copia la ruta exacta**: la vas a necesitar en el siguiente paso.

CUIDADO

Si ya tenías otra versión de Java instalada (por ejemplo, la de Oracle), puede haber conflicto. Oracle agrega al Path la ruta `C:\Program Files\Common Files\Oracle\Java\javapath`. En el capítulo 3 verás cómo dar prioridad a tu JDK nuevo.

ALTERNATIVA

También puedes instalar el JDK desde la terminal con `winget`, el gestor de paquetes de Windows:

```
PS> winget install EclipseAdoptium.Temurin.25.JDK
```

CAPÍTULO 03

3 CONFIGURAR JAVA_HOME Y PATH

> Las **variables de entorno** le dicen a Windows dónde buscar programas. Configurarlas te permite escribir `java` o `javac` desde cualquier carpeta.

- **JAVA_HOME**: apunta a la carpeta raíz del JDK. Muchas herramientas (Maven, Gradle, Spring Boot) la buscan.
- **Path**: es la lista de carpetas donde Windows busca ejecutables. Le agregaremos la carpeta `bin` del JDK.

Paso 1 · Abrir las variables de entorno

1. Pulsa la tecla `⊞ Win` y escribe **"variables de entorno"**. Abre **Editar las variables de entorno del sistema**.
2. En la ventana **Propiedades del sistema**, haz clic en el botón **Variables de entorno…**

Paso 2 · Crear JAVA_HOME

3. En la sección inferior, **Variables del sistema**, pulsa **Nueva...**

4. Escribe:
   Nombre de la variable: `JAVA_HOME`
   Valor de la variable: `C:\Program Files\Eclipse Adoptium\jdk-25.0.x-hotspot`
   Usa la ruta real que copiaste, **sin** `\bin` al final. Pulsa **Aceptar**.

Paso 3 · Agregar el JDK al Path

5. En **Variables del sistema**, selecciona la variable `Path` y pulsa **Editar...**

6. Pulsa **Nuevo** y escribe exactamente: `%JAVA_HOME%\bin`

7. Con esa línea seleccionada, pulsa **Subir** hasta que quede **de primera**. Así tiene prioridad sobre otras versiones de Java.

8. Pulsa **Aceptar** en las tres ventanas para guardar los cambios.

Paso 4 · Verificar

Abre una terminal **nueva** (CMD o PowerShell; las que ya estaban abiertas no ven los cambios) y escribe:

```
C:\> java -version
openjdk version "25.0.x" ... LTS
C:\> javac -version
javac 25.0.x
C:\> echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-25.0.x-hotspot
```

SI ALGO FALLA

"java no se reconoce como un comando interno o externo": revisa que `%JAVA_HOME%\bin` esté en el Path y que abriste una terminal nueva.
Aparece otra versión: sube `%JAVA_HOME%\bin` al primer lugar del Path.
En PowerShell, para ver la variable usa `$env:JAVA_HOME` en vez de `echo %JAVA_HOME%`.

CAPÍTULO 04

4 VISUAL STUDIO CODE PARA JAVA

> VS Code es un editor de código gratuito, liviano y muy popular. Con las extensiones correctas se convierte en un entorno completo para Java.

Instalación

1. Descarga VS Code desde `code.visualstudio.com` y ejecuta el instalador.
2. En **Tareas adicionales** marca **"Agregar a PATH"** y las opciones **"Abrir con Code"**. Así podrás abrir carpetas con clic derecho.
3. Abre VS Code, ve a Extensiones (`Ctrl` + `Shift` + `X`), busca **Extension Pack for Java** (de Microsoft) e instálalo. Incluye soporte del lenguaje, depurador, ejecución de pruebas y gestión de proyectos.

Tu primer programa

4. Crea una carpeta, por ejemplo `clase-01`, y ábrela en VS Code: **Archivo → Abrir carpeta…**
5. Crea un archivo nuevo llamado `HolaMundo.java`. **El nombre del archivo debe ser igual al nombre de la clase**, respetando mayúsculas.
6. Escribe el siguiente código y pulsa el botón **▶ Run** que aparece sobre el método `main`, o en la esquina superior derecha.

```
HolaMundo.java
public class HolaMundo {
    public static void main(String[] args) {
        // Esto imprime un mensaje en la consola
        System.out.println("¡Hola, CI Nicaragua!");
    }
}
```

Anatomía del programa

Parte	Significado
`public class HolaMundo`	Declara una clase. En Java todo el código vive dentro de clases.
`public static void main(...)`	El **punto de entrada**: es lo primero que ejecuta la JVM.
`System.out.println(...)`	Imprime texto en la consola y salta de línea.
`{ }` y `;`	Las llaves agrupan bloques; el punto y coma termina cada instrucción.

También desde la terminal

Abre la terminal integrada con `Ctrl`+`ñ` en teclado español (o **Terminal → Nueva terminal**) y compila y ejecuta a mano. Así ves el proceso completo del capítulo 1:

```
> javac HolaMundo.java      # crea HolaMundo.class
> java HolaMundo            # la JVM lo ejecuta
¡Hola, CI Nicaragua!
```

ATAJOS ÚTILES

`Ctrl`+`Shift`+`P` paleta de comandos · `Shift`+`Alt`+`F` dar formato al código · `Ctrl`+`Espacio` autocompletar · escribe `sysout` + `Tab` para generar `System.out.println()`.

CAPÍTULO 05

5 VARIABLES Y TIPOS DE DATOS

> Una **variable** es un espacio en memoria con un nombre, donde guardas un valor que puede cambiar. En Java, toda variable tiene un **tipo** que define qué puede guardar.

```
tipo nombre = valor;

int edad = 17;
String nombre = "Ana";
```

Tipos primitivos

Java tiene 8 tipos primitivos: los bloques básicos de datos.

Tipo	Tamaño	Guarda	Ejemplo
`byte`	8 bits	Enteros de −128 a 127	`byte nivel = 5;`
`short`	16 bits	Enteros de −32 768 a 32 767	`short anio = 2026;`
`int`	32 bits	Enteros de unos ±2 100 millones	`int edad = 17;`
`long`	64 bits	Enteros muy grandes (sufijo `L`)	`long pob = 7000000L;`
`float`	32 bits	Decimales, ~7 dígitos (sufijo `f`)	`float nota = 85.5f;`
`double`	64 bits	Decimales, ~15 dígitos	`double precio = 99.95;`
`char`	16 bits	Un solo carácter, comillas simples	`char letra = 'A';`
`boolean`	—	Solo `true` o `false`	`boolean activo = true;`

EN LA PRÁCTICA

El 90 % del tiempo usarás `int` para enteros, `double` para decimales, `boolean` para condiciones y `String` para texto.

String, var y constantes

`String` no es primitivo: es una **clase** para manejar texto, por eso se escribe con mayúscula y va entre comillas dobles. Con `var` Java deduce el tipo por ti, y con `final` creas una **constante** que no puede cambiar.

Variables.java

```java
public class Variables {
    public static void main(String[] args) {
        String curso = "Backend con Java";
        int horas = 30;
        double promedio = 87.5;
        boolean aprobado = promedio >= 70;
        var temporada = "XIV";            // Java deduce que es String
        final double IVA = 0.15;          // constante: no puede cambiar

        System.out.println("Curso: " + curso + " (" + horas + " h)");
        System.out.println("¿Aprobado? " + aprobado);
    }
}
```

Reglas para nombrar variables

- Empiezan con letra, `_` o `$`, nunca con número.
- Sin espacios ni tildes; Java distingue mayúsculas: `edad` ≠ `Edad`.
- No pueden ser palabras reservadas (`class`, `int`…).
- Variables en **camelCase**: `precioTotal`.
- Constantes en **MAYÚSCULAS**: `TASA_IVA`.
- Clases en **PascalCase**: `HolaMundo`.

Operadores básicos

Tipo	Operadores	Ejemplo
Aritméticos	`+ - * / %`	`7 / 2` → `3` (entre enteros) · `7 % 2` → `1`
Comparación	`== != > < >= <=`	`edad >= 18` → `true` o `false`
Lógicos	`&&` (y) · `||` (o) · `!` (no)	`edad >= 15 && activo`
Asignación	`= += -= ++ --`	`contador++;` suma 1

CAPÍTULO 06

6 ESTRUCTURAS DE CONTROL

> Por defecto, un programa se ejecuta línea por línea, de arriba hacia abajo. Las **estructuras de control** permiten tomar decisiones y repetir acciones.

Decisiones: if, else if, else

Ejecuta un bloque de código solo si una condición es verdadera.

```
int nota = 82;

if (nota >= 90) {
    System.out.println("Excelente");
} else if (nota >= 70) {
    System.out.println("Aprobado");      // se imprime esta
} else {
    System.out.println("Reprobado");
}
```

Decisiones múltiples: switch

Útil cuando comparas una variable con varios valores fijos. Desde Java 14 existe la forma con flecha `->`, más corta y sin necesidad de `break`:

```
int dia = 2;
String nombre = switch (dia) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    case 4 -> "Jueves";
    default -> "Otro día";
};
System.out.println("Hoy es " + nombre);   // Hoy es Martes
```

Repetición: for

Se usa cuando sabes **cuántas veces** repetir. Tiene tres partes: inicio; condición; incremento.

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Clase número " + i);
}
```

Repetición: while y do-while

`while` repite **mientras** la condición sea verdadera; puede no ejecutarse nunca. `do-while` ejecuta el bloque **al menos una vez** y después revisa la condición.

while

```java
int intentos = 3;
while (intentos > 0) {
    System.out.println(intentos);
    intentos--;
}
```

do-while

```java
int n = 10;
do {
    System.out.println(n);
    n++;
} while (n < 5);  // imprime 10
```

break y continue

`break` sale del ciclo inmediatamente; `continue` salta a la siguiente vuelta.

```java
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   // ignora los pares
    if (i > 7) break;           // se detiene después del 7
    System.out.print(i + " ");  // 1 3 5 7
}
```

CICLO INFINITO

Si la condición de un `while` nunca se vuelve falsa, el programa no termina. Asegúrate de que algo cambie dentro del ciclo (como `intentos--`). Para detener un programa colgado en la terminal pulsa `Ctrl` + `C`.

CAPÍTULO 07

7 PARA PRACTICAR

Resuelve estos ejercicios en tu carpeta `clase-01`. Crea un archivo `.java` por ejercicio y súbelos a tu repositorio.

01 **Verifica tu entorno** básico

Toma una captura de la terminal mostrando el resultado de `java -version`, `javac -version` y la variable `JAVA_HOME`.

02 **Tarjeta de presentación** básico

Declara variables con tu nombre, edad, ciudad y si ya habías programado antes (`boolean`). Imprímelas en un formato ordenado.

03 **Calculadora de promedio** intermedio

Declara tres notas tipo `double`, calcula el promedio y muestra “Aprobado” si es mayor o igual a 70, o “Reprobado” en caso contrario.

04 **Tabla de multiplicar** `intermedio`

Usa un ciclo `for` para imprimir la tabla del 7, del 7 × 1 al 7 × 10.

05 **Días de clase** `intermedio`

Con un `switch`, recibe un número del 1 al 7 y muestra el día de la semana. Si es 2 o 4, agrega "¡Hay clase de Java!".

06 **Suma hasta 100** `reto`

Con un `while`, suma los números 1, 2, 3… hasta que la suma supere 100. Muestra cuántos números sumaste y el total.

Antes de la próxima clase

- Tener el JDK 25 y VS Code instalados y funcionando.
- Poder explicar con tus palabras la diferencia entre JDK, JRE y JVM.
- Haber ejecutado al menos un programa con variables y un ciclo.

Referencias

Oracle. *The Java™ Tutorials*. docs.oracle.com/javase/tutorial

Eclipse Adoptium. *Temurin Releases*. adoptium.net

Microsoft. *Getting Started with Java in VS Code*. code.visualstudio.com/docs/java



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno · Temporada XIV · 2026